

Mobilni objekti koji putuju mrežom (međusobno isključivanje distribuiranih procesa)

Filip Matasić¹, Marina Matešić¹, and Tomislav Novak¹

¹Prirodoslovno-matematički fakultet

U ovom radu obrađujemo algoritme za međusobno isključivanje procesa u distribuiranom sustavu. Rad je projekt iz kolegija *Distribuirani sustavi* na Prirodoslovno-matematičkom fakultetu u ak. god. 2025./2026., koji vodi prof. dr. sc. Robert Manger.

1 Uvod i općenito o distribuiranim procesima

Distribuirani sustav je sustav više procesora čiji je jedini način komunikacije razmjenom poruka preko neke zajedničke komunikacijske mreže. Svaki procesor ima samo svoju lokalnu memoriju.

Program je niz naredbi u višem programskom ili strojnom jeziku, a *proces* je produkt pokretanja programa.

2 Problem međusobnog isključivanja

Problem među distribuiranim procesima javlja se kada oni trebaju pristupiti nekom *dijeljenom resursu*. Naime, lako je vidjeti da bi se mogle dogoditi logičke pogreške i neispravna ažuriranja memorije ako njoj pristupaju dva ili više procesa istovremeno. Zato se pristup tom dijeljenom resursu mora omogućiti najviše jednom procesu u svakom trenutku, i time se bavi *problem međusobnog isključivanja*. Rad procesa s tim konfliktnim dijeljenim resursom naziva se *kritična sekcija* (u drugoj literaturi: *kritični odsječak*) tog procesa.

Navedimo glavne zahtjeve koje problem međusobnog isključivanja mora riješiti:

- *sigurnost* (ne smije se dogoditi da dva procesa istovremeno pristupaju kritičnoj sekciji)
- *odsustvo izgladnjivanja* (proces koji zatraži ulazak u kritičnu sekciju kad-tad će ga i dobiti)
- *pravednost* (zahtjevi za ulazak u kritični odsječak moraju se obrađivati u redosljedu koji ne favorizira nijedan proces)

U skripti s predavanja Manger (2017–2025) bavili smo se dvjema vrstama algoritama za međusobno isključivanje:

- algoritmi zasnovani na vremenskim žigovima
- algoritmi zasnovani na žetonima (*token*)

U ovom radu obrađujemo tri algoritma dana u knjizi Raynal (2013), a zasnovani su na *mobilnim objektima*. Mobilne objekte može se smatrati poopćenjem žetona, budući da je žeton samo fiktivan objekt bez informacija, a mobilni objekt pri kruženju mrežom služi i kao poruka, to jest vrsta zapisa odn. plutajućeg komadića memorije kojoj pristupa svaki proces kad ga zaprimi. Isto kao i kod žetona, mobilni objekt smije se u svakom trenutku nalaziti kod najviše jednog procesa (ili kod jednog procesa, ili negdje na putu u mreži).

3 Algoritmi i tehnički detalji

U sljedećim odlomcima opisujemo tri algoritma koji upravljaju putovanjem mobilnog objekta:

- *Algoritam putovanja mobilnog objekta u potpunoj mreži* (A Navigation Algorithm for a Complete Network)
- *Algoritam putovanja mobilnog objekta u mreži zasnovan na razapinjućem stablu* (A Navigation Algorithm Based on a Spanning Tree)
- *Prilagođeni algoritam putovanja mobilnog objekta* (An Adaptive Navigation Algorithm)

Kod je priložen u *GitHub* repozitoriju Tomislav Novak (2025), a implementiran je u programskom jeziku Java. Kod je zapravo nadopuna *frameworka* danog kao nastavni materijal, a autora V. K. Garga i dostupnog kao suplementarni materijal priložen Garg (2004). Dodane su i implementirane klase potrebne za pripremu tri spomenuta algoritma, i blago izmijenjena klasa *LockTester* samo radi mogućnosti pozivanja novih klasa.

U svim algoritmima pretpostavljamo N procesa $p_1, \dots, p_N$.

4 Algoritam putovanja mobilnog objekta u potpunoj mreži

4.1 Algoritam

U ovom algoritmu, umjesto *golog* žetona, mobilni objekt sadrži cjelobrojnu listu *obtained* duljine N . Ona na i -tom mjestu sadrži informaciju koliko puta je proces p_i primio objekt. Jasno, inicijalizirana je nul-vrijednostima.

Svaki proces još lokalno drži sljedeće informacije:

- Lista *request_by*, u kojoj na j -tom mjestu stoji koliko je taj proces od procesa p_j dotad primio zahtjeva za ulazak u kritičnu sekciju. Inicijalizirana je nul-vrijednostima.
- Bool *wantCS* (*want critical section*) - zastavica želi li taj proces ući u kritičnu sekciju. Inicijaliziran je na *false*.

- Bool `haveObject` - zastavica je li objekt kod tog procesa. Inicijalizirana na *false*, osim kod procesa koji prvi ima objekt gdje je *true*, a u našoj implementaciji to je uvijek proces p_1 .

Neka je u nekom trenutku objekt kod procesa p_i . Primijetimo da je tada njemu lako odrediti koji sve procesi čekaju na ulazak u kritičnu sekciju (žele objekt) - to su oni s indeksima j na kojima vrijedi `request_by[j] > obtained[j]`, tj. oni procesi koji su strogo više puta zatražili objekt nego što su ga dobili.

Navodimo pseudokod algoritam i zatim ga komentiramo.

Algorithm 1: Algoritam putovanja mobilnog objekta u potpunoj mreži (za proces p_i)

```

1 function acquire_object() :
2   wantCS  $\leftarrow$  true
3   if not haveObject then
4     request_by[i]  $\leftarrow$  request_by[i] + 1
5     for each  $k \in \{1, \dots, n\} \setminus \{i\}$  do
6       send REQUEST(i) to process  $p_k$ 
7     wait while haveObject  $\neq$  true

8 function release_object() :
9   wantCS  $\leftarrow$  false
10  obtained[i]  $\leftarrow$  request_by[i]
11  for  $k$  from  $i + 1$  to  $n$  then 1 to  $i - 1$  do
12    if request_by[k] > obtained[k] then
13      haveObject  $\leftarrow$  false
14      send OBJECT() to process  $p_k$ 
15    return

16 received OBJECT() haveObject  $\leftarrow$  true

17 received REQUEST( $k$ ) request_by[k]  $\leftarrow$  request_by[k] + 1
18 if haveObject and not wantCS then
19   haveObject  $\leftarrow$  false
20   send OBJECT() to process  $p_k$ 

```

Jasno, kad proces primi objekt, samo postavi zastavicu da ga je primio.

Kad primi zahtjev za ulaskom u kritičnu sekciju od procesa p_k , ažurira to u listi zahtjeva `request_by`; te, ako ima objekt i sam ne želi ući u kritičnu sekciju, šalje ga procesu p_k .

Kad proces želi ući u kritičnu sekciju zove funkciju `acquire_object()` (u kodu: `requestCS`, da bude analogno s ostalim algoritmima), koja postavi pripadnu lokalnu zastavicu. Zatim, ako nema objekt, traži ga (što se sastoji od slanja zahtjeva svim ostalim procesima, i ažuriranja u lokalnoj listi zahtjeva) i zatim čeka da ga primi (pauzira dretvu).

Kad proces završi s kritičnom sekcijom zove funkciju `release_object()` (u kodu: `releaseCS`), koja, opet, postavi pripadnu lokalnu zastavicu. Tada u objektu prepíše vrijednost `obtained` pripadnog procesa pripadnom vrijednosti iz `request_by` (da se zna da je ispunio svoje dotadašnje zahtjeve). Naposljetku traži ima li kandidata koji traži ulazak u kritičnu sekciju (kako je navedeno prije algoritma), i to kružnim redom. Ako ima, šalje mu objekt.

Provjerimo jesu li ispunjeni zahtjevi problema međusobnog isključivanja.

- *Sigurnost*. Objekt je u svakom trenutku kod najviše jednog procesa, a proces ne ulazi u kritičnu sekciju dok nema zastavicu da je objekt kod njega.
- *Odsustvo izgladnjivanja*. Osigurano je kružnom provjerom na sljedeće kandidate za proslijediti objekt (svaki će objekt čekati najvulje da svih ostalih $N - 1$ objekata uđe u kritičnu sekciju i onda će on doći na red).
- *pravednost*. Ovo svojstvo nije ispunjeno; primjerice, dok proces p_1 obavlja kritičnu sekciju stižu mu zahtjevi od redom p_3 i p_2 , no on po izlasku iz kritične sekcije objekt šalje procesu p_2 jer tim redom provjerava želje.

References

- Vijay K. Garg. *Concurrent and Distributed Computing in Java*. Wiley, 2004. URL <https://users.ece.utexas.edu/~garg/jbk.html>. Pristupljeno 15. 5. 2025.
- Leonardo Jelenković. Operacijski sustavi (skripta), 2024. Pristupljeno 15. 5. 2024. na <https://www.zemris.fer.hr/~leonardo/os/math/predavanja/>.
- Robert Manger. Distribuirani procesi (nastavni materijal), 2017–2025. Pristupljeno 15. 5. 2025. na <http://web.studenti.math.pmf.unizg.hr/~manger/dp/>.
- Michel Raynal. *Distributed Algorithms for Message-Passing Systems*. Springer, 2013. Pristupljeno 15. 5. 2025.
- Tomislav Novak. dp2025 (repozitorij), 2025. URL <https://github.com/tnov8/dp2025>.